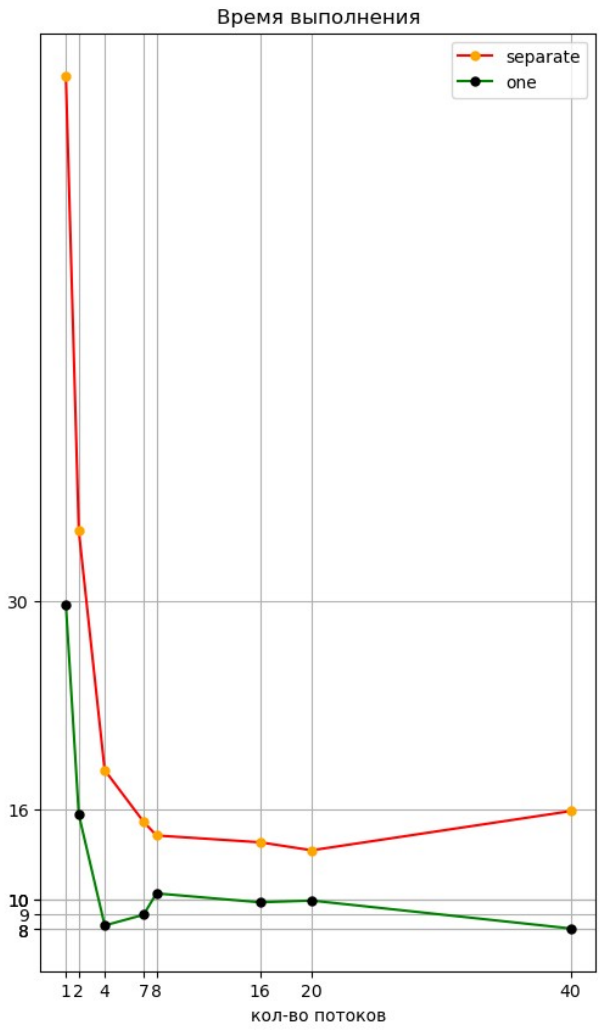
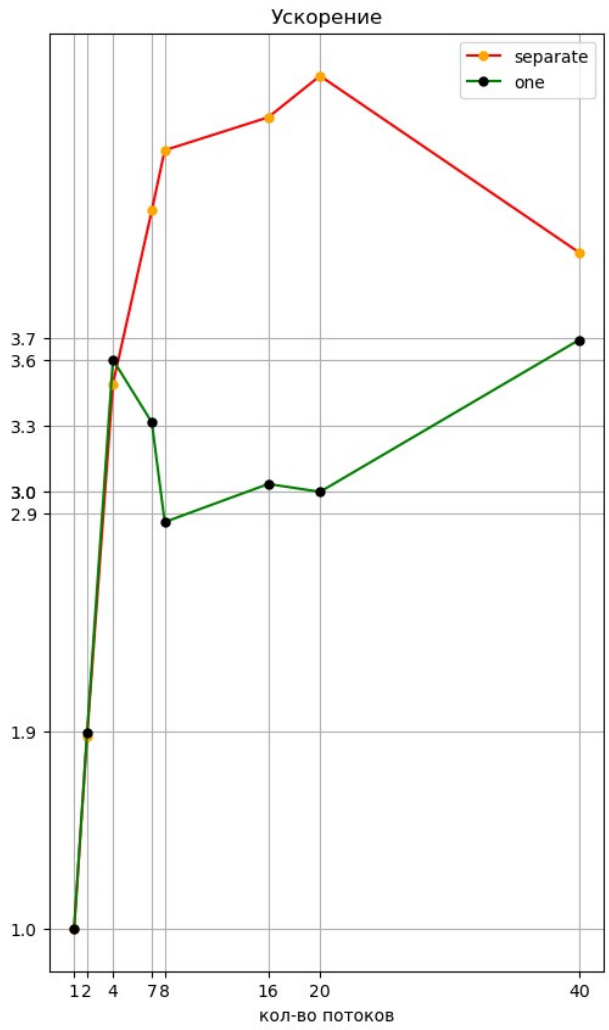
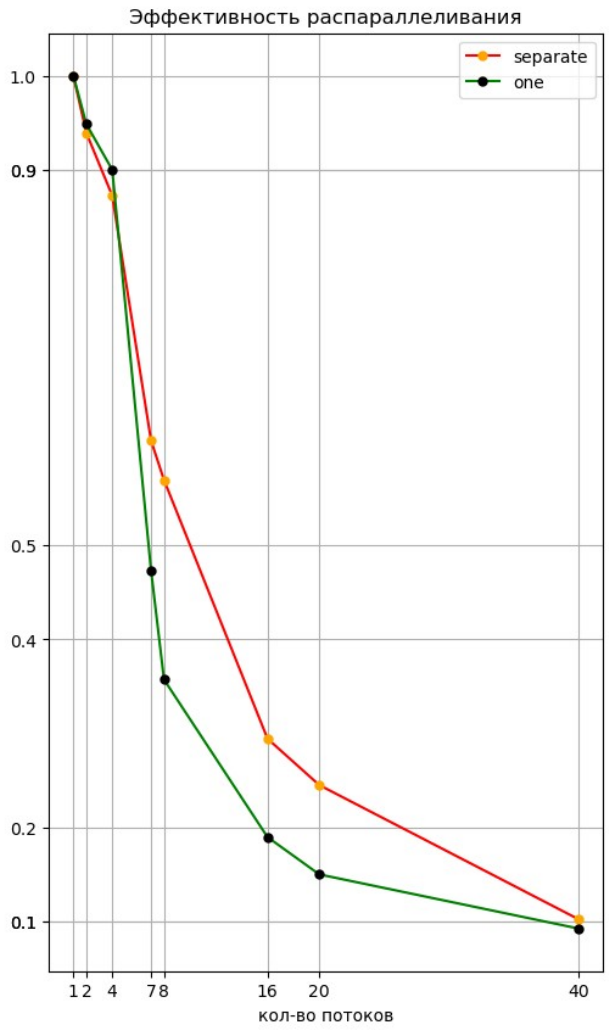
# Замеры времени и подсчет коэффициента ускорения и эффективности

| | Количество потоков | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | 2 | | 4 | | 7 | | 8 | | 16 | | 20 | | 40 |
| N = 1600, eps=1e-5, max_iter=15000 | T_1 | T_2 | S_2 | T_4 | S_4 | T_7 | S_7 | T_8 | S_8 | T_16 | S_16 | T_20 | S_20 | T_40 | S_40 |
| Вариант 1 | 65.103 | 34.656 | 1.88 | 18.654 | 3.49 | 15.205 | 4.28 | 14.284 | 4.56 | 13.827 | 4.71 | 13.292 | 4.9 | 15.913 | 4,09 |
| Вариант 2 | 29.75 | 15.681 | 1.9 | 8.264 | 3.6 | 8.977 | 3.31 | 10.409 | 2.86 | 9.812 | 3,03 | 9.929 | 3.9 | 8.062 | 3.69 |
| | | | | | | | | | | | | | | | |
| Время выполнения: | Eff_1 | | Eff_2 | | Eff_4 | | Eff_7 | | Eff_8 | | Eff_16 | | Eff_20 | | Eff_40 |
| Вариант 1 | 1 | | 0.94 | | 0.87 | | 0.61 | | 0.57 | | 0.29 | | 0.24 | | 0.1 |
| Вариант 2 | 1 | | 0.95 | | 0.9 | | 0.47 | | 0.36 | | 0.19 | | 0.15 | | 0.09 |

# Графики зависимостей



# Статистика для omp parallel for schedule()

| schedule type, N = 1600, eps = 1e-5, threads=4, max_iter=15000 | static | static, 1 | static, 32 | static, 128 | dynamic | dynamic, 8 | dynamic, 32 | guided | guided, 16 | auto |
|---|---|---|---|---|---|---|---|---|---|---|
| Время выполнения: | 22.78 | 22.68 | 22.56 | 23.02 | 23.11 | 23.59 | 20.23 | 17.985 | 17.982 | 17.974 |

## Вывод о выборе метода распараллеливания:

Выполнив лабораторную работу №2 задание 3 наиболее предпочтительный способ распараллеливания — это **вариант №2** (*Одна большая секция #pragma omp parallel с ручным распределением итераций*)

**Анализ графиков и таблицы:**

- **Абсолютное время выполнения** у **Варианта 2** значительно ниже почти на всём диапазоне (от 2 до 40 потоков). Например, при 40 потоках время составляет **8.06 с** против **15.91 с** у Варианта 1.
- **Ускорение** у Варианта 1 выглядит выше на отдельных точках, но это не сильно решает — оно считается от более медленного последовательного времени (65 с). Реальное преимущество по скорости - за Вариантом 2.
- **Эффективность распараллеливания** у обоих вариантов падает с ростом числа потоков (ибо наша задача является memory-bound), однако Вариант 2 показывает более плавное и стабильное снижение, однако оно более крутое, чем у Варианта 1 (из-за меньшего T1 у Варианта 2)

Причины выбора **варианта №2:**

- *Минимальный overhead — потоки создаются только один раз, а не по 2 раза (в худшем случае 15000итер. x2) как в Варианте 1.*
- *Лучшее время выполнения по большей части потоков, по сравнению с Вариантом 1.*

- *После 4 потоков кажется, что Вариант 1 выигрывает за счет ускорения и эффективности распараллеливания, однако это необъективно, так как из-за двух циклов создания потоков время выполнения T1 на одном потоке значительно больше, чем у Варианта 2. Вариант 2 все равно выигрывает преимущественно за счёт скорости выполнения программы*

**Примечание к замерам времени у schedule:** С одинаковыми начальными данными и параметрами значения времени выполнения программы может отличаться на 15-30% из-за недетерминированности распределения потоков планировщиком ОС и из-за борьбы потоков за RAM и доступ к кэшу (так как задача memory-bound)